Environmental PCA Tutorial

This is a tutorial using R and QGIS to acquire environmental variable rasters, get values for those variables at species occurrence sites, and perform a principal component analysis on those variables. Each individual part can be used separately, if you're just looking to download environmental variables or just looking to run a PCA. I hope you enjoy playing around with this code as much as I enjoyed putting it together. Feel free to share with anyone you think could make use of it. Let me know if you have any questions!

-Rachel F. Kruger rkruger1@binghamton.edu

1. Prepare your occurrence data files

You should have a csv file with coordinates of your populations. It should be set up with the first column being **species** (or group), the second column being **longitude** and the third column being **latitude**

2. Download Environmental Variables

1. Install necessary packages

```
# Load in packages
library(sf)
library(terra)
library(raster)
library(geodata)
library(ENMTools)
library(prism)
```

2. Bioclim Variables

Adapted from: https://rsh249.github.io/spatial_bioinformatics/worldclim.html For more information on Bioclim variables, please see: https://www.worldclim.org/data/bioclim.html

For more information on the **geodata** package, visit the CRAN site https://cran.r-project.org/web/packages/geodata/index.html or the GitHub page https://github.com/rspatial/geodata

a. Download climate tiles

30 arc-second resolution is the finest resolution you can get for Bioclim variables. To get this, you need to download by tile. You will make one object for each corner of your extent. Northwest, Northeast, Southwest, Southeast. Be sure the extent encompasses all population occurrence points, but try to minimize the extent to minimize disk space used.

You will use the function *worldclim_tile* from the **geodata** package is the function to use for this. You can alternatively use worldclim to get data for an entire country or the globe. You will download four tiles: one for

each corner of your geographic extent.

```
# env <- worldclim_tile(var = "bio" for all 19 bioclim variables
# lon=west point/east point
# lat=north point/south point
# path="directory you want to save")

#Note: This may take several minutes

env_northwest <- worldclim_tile(var = "bio", lon=-121, lat=37.1, path =
"YOUR_PATH")

env_northeast <- worldclim_tile(var = "bio", lon=-115, lat=37.1, path =
"YOUR_PATH")

env_southwest <- worldclim_tile(var = "bio", lon=-121, lat=30, path = "YOUR_PATH")

env_southeast <- worldclim_tile(var = "bio", lon=-115, lat=30, path = "YOUR_PATH")
```

b. Merge tiles into a RasterStack

Using the *merge* function from the **raster** package to merge the first two, then that one to the third, then that one to the fourth to create a RasterStack with the data for a square of the rough extent you specified.

```
env12<-merge(env_northwest, env_northeast)

env123<-merge(env12, env_southwest)

env_all<-merge(env123, env_southeast)
```

c. Mask the raster tile to the extent you want

This just makes it so the final raster you save isn't so large. The tiles that you download are larger than the extent you specified - they are just *at least* encompass the coordinates you set.

```
ext <- as(extent(-121, -115, 30, 37.1), 'SpatialPolygons')

bio <- crop(env_all, extent(st_bbox(ext)))
```

d. Write raster files to your computer

```
# Create an output folder object where you want the raster files to be written to.
output_folder <- "YOUR_PATH"

for (i in 1:nlyr(bio)){
```

```
  output_filename <- file.path(output_folder, paste0(names(bio)[i], ".tif"))
  writeRaster(bio[[i]], filename = output_filename, overwrite=T)

# This just tells you whether the rasters contain NA values. Not actually
necessary
if (any(is.na(values(bio[[i]]))) | any(!is.finite(values(bio[[i]])))) {
  warning(paste("Layer", names(bio)[i], "contains NA or non-finite values"))
}

# This will plot each Bioclim layer one at a time to show you what your raster
will look like
  plot(bio[[i]], main = paste("Layer", names(bio)[i]))
}
```

3. ISRIC (Soil) Data

ISRIC: International Soil Reference and Information Centre

ISRIC World Soil Information gives us access to rasters of soil data in both continuous and categorical variables. To see what the soil data variables are, once you've loaded the **geodata** package to your library, use "?soil_world" to access the help file that describes the different variables. For more information on the **geodata** package, visit the CRAN site https://cran.r-project.org/web/packages/geodata/index.html or the GitHub page https://github.com/rspatial/geodata

For more information on ISRIC data, please see: https://www.isric.org/

*Note: I have not figured out how to properly download categorical variables such as soil type.

a. Download continuous soil variables

You'll use the *soil_world* function to download each of the variables you want. Change the 'depth' argument to access different soil depths. See "?soil_world" for details on the depth argument as well as what each variable is.

List of continuous variables:

- bdod: Bulk density of the fine earth fraction (kg dm-3)
- cec: Cation Exchange Capacity of the soil (cmol(+) kg-1)
- cfvo: Vol. fraction of coarse fragments (> 2 mm) (%)
- nitrogen: Total nitrogen (N) (g kg-1)
- phh2o: pH (H2O)
- sand: Sand (> 0.05 mm) in fine earth (%)
- silt: Silt (0.002-0.05 mm) in fine earth (%)
- clay: Clay (< 0.002 mm) in fine earth (%)
- soc: Soil organic carbon in fine earth (g kg-1)

- ocd: Organic carbon density (kg m-3)
- ocs: Organic carbon stocks (kg m-2)

```
bdod <- soil_world(var="bdod", depth=5, path='YOUR_PATH')

cfvo <- soil_world(var="cfvo", depth=5, path='YOUR_PATH')

clay <- soil_world(var="clay", depth=15, path='YOUR_PATH')

nitrogen <- soil_world(var="nitrogen", depth=5, path='YOUR_PATH')

ocd <- soil_world(var="ocd", depth=5, path='YOUR_PATH')

phh2o <- soil_world(var="phh2o", depth=5, path='YOUR_PATH')

sand <- soil_world(var="sand", depth=5, path='YOUR_PATH')

silt <- soil_world(var="silt", depth=5, path='YOUR_PATH')

soc <- soil_world(var="soc", depth=5, path='YOUR_PATH')

elev <- elevation_30s(country = "USA", path = 'YOUR_PATH')
```

b. Create a polygon with the extent you want to crop your rasters to.

```
e <- as(extent(-121, -115, 30, 37.1), 'SpatialPolygons')

#Set crs of the extent
crs(e) <- "+proj=longlat +datum=WGS84 +no_defs"
```

c. Crop variable rasters to your chosen extent

```
# Crop original variable raster to the extent
cbdod <- crop(bdod, extent(st_bbox(e)))

# Now you need to rasterize it using the "raster" function
b <- raster(cbdod)

# You can plot the first raster to see if it looks right
plot(b)

# Now do it for all other variables
ccfvo <- crop(cfvo, extent(st_bbox(e)))
cf <- raster(ccfvo)

cclay <- crop(clay, extent(st_bbox(e)))
cl <- raster(cclay)
```

```
cnitrogen <- crop(nitrogen, extent(st_bbox(e)))
n <- raster(cnitrogen)

cocd <- crop(ocd, extent(st_bbox(e)))
o <- raster(cocd)

cphh2o <- crop(phh2o, extent(st_bbox(e)))
p <- raster(cphh2o)

csand <- crop(sand, extent(st_bbox(e)))
sa <- raster(csand)

csilt <- crop(silt, extent(st_bbox(e)))
si <- raster(csilt)

csoc <- crop(soc, extent(st_bbox(e)))
so <- raster(csoc)

celev <- crop(elev, extent(st_bbox(e)))
el <- raster(celev)
```

d. Write each raster as a GeoTIFF file on your computer

```
writeRaster(b, filename = "YOUR_PATH/bdod.tif", format="GTIFF", overwrite=T)

writeRaster(cf, filename = "YOUR_PATH/cfvo.tif", format="GTIFF", overwrite=T)

writeRaster(cl, filename = "YOUR_PATH/clay.tif", format="GTIFF", overwrite=T)

writeRaster(n, filename = "YOUR_PATH/nitrogen.tif", format="GTIFF", overwrite=T)

writeRaster(o, filename = "YOUR_PATH/ocd.tif", format="GTIFF", overwrite=T)

writeRaster(p, filename = "YOUR_PATH/phh2o.tif", format="GTIFF", overwrite=T)

writeRaster(sa, filename = "YOUR_PATH/sand.tif", format="GTIFF", overwrite=T)

writeRaster(si, filename = "YOUR_PATH/silt.tif", format="GTIFF", overwrite=T)

writeRaster(so, filename = "YOUR_PATH/soc.tif", format="GTIFF", overwrite=T)

writeRaster(el, filename = "YOUR_PATH/elev.tif", format="GTIFF", overwrite=T)
```

4. PRISM Data

PRISM Data shows various climatological variables. More information here: https://prism.oregonstate.edu/ We are using the package **prism** to do this. For more information on this package, visit the CRAN site

https://cran.r-project.org/web/packages/prism/index.html or the GitHub site https://github.com/ropensci/prism for vignettes on the different functionalities of the package.

a. Set download directory

```
# Create a download directory
prism_set_dl_dir('YOUR_PATH', create = T)
```

b. Download PRISM variables

In this tutorial I'm showing you how to get 30-year normals (1991-2020), but you use different functions to get annual (*get_prism_annual*), monthly(*get_prism_monthlys*), or single-day values (*get_prism_dailys*).

There are two resolution options: 4km and 800m. We'll be using the higher resolution (800m). The variables available for PRISM are:

- ppt: precipitation
- tmean: Mean temperature (mean of tmin and tmax)
- tmin: Minimum temperature
- tmax: Maximum temperature
- tdmean: Mean dew point temperature
- vpdmin: Minimum vapor pressure deficit
- vpdmax: Maximum vapor pressure deficit
- solclear: Solar radiation (clear sky) *Only available for 30 year normals
- solslope: Solar radiation (sloped) *Only available for 30 year normals
- soltotal: Total Solar radiation *Only available for 30 year normals
- soltrans: Cloud transmittance *Only available for 30 year normals

```
ppt <- get_prism_normals(type = "ppt", resolution = "800m", mon = NULL, annual =
T, keepZip = F)

tmean <- get_prism_normals(type = "tmean", resolution = "800m", mon = NULL, annual
= T, keepZip = F)

tmin <- get_prism_normals(type = "tmin", resolution = "800m", mon = NULL, annual =
T, keepZip = F)

tmax <- get_prism_normals(type = "tmax", resolution = "800m", mon = NULL, annual =
T, keepZip = F)

tdmean <- get_prism_normals(type = "tdmean", resolution = "800m", mon = NULL,
```

```
annual = T, keepZip = F)

vpdmin <- get_prism_normals(type = "vpdmin", resolution = "800m", mon = NULL,
annual = T, keepZip = F)

vpdmax <- get_prism_normals(type = "vpdmax", resolution = "800m", mon = NULL,
annual = T, keepZip = F)

solclear <- get_prism_normals(type = "solclear", resolution = "800m", mon = NULL,
annual = T, keepZip = F)

solslope <- get_prism_normals(type = "solslope", resolution = "800m", mon = NULL,
annual = T, keepZip = F)

soltotal <- get_prism_normals(type = "soltotal", resolution = "800m", mon = NULL,
annual = T, keepZip = F)

soltrans <- get_prism_normals(type = "soltrans", resolution = "800m", mon = NULL,
annual = T, keepZip = F)
```

c. Rasterize each variable

```
ppt1 <-
raster("YOUR_PATH/PRISM_ppt_30yr_normal_800mM4_annual_bil/PRISM_ppt_30yr_normal_80
0mM4_annual_bil.bil")
crs(ppt1) <- st_crs(4326)$wkt

tmean1 <-
raster("YOUR_PATH/PRISM_tmean_30yr_normal_800mM5_annual_bil/PRISM_tmean_30yr_norma
l_800mM5_annual_bil.bil")
crs(tmean1) <- st_crs(4326)$wkt

tmin1 <-
raster("YOUR_PATH/PRISM_tmin_30yr_normal_800mM5_annual_bil/PRISM_tmin_30yr_normal_
800mM5_annual_bil.bil")
crs(tmin1) <- st_crs(4326)$wkt

tmax1 <-
raster("YOUR_PATH/PRISM_tmax_30yr_normal_800mM5_annual_bil/PRISM_tmax_30yr_normal_
800mM5_annual_bil.bil")
crs(tmax1) <- st_crs(4326)$wkt

tdmean1 <-
raster("YOUR_PATH/PRISM_tdmean_30yr_normal_800mM5_annual_bil/PRISM_tdmean_30yr_nor
mal_800mM5_annual_bil.bil")
crs(tdmean1) <- st_crs(4326)$wkt

vpdmin1 <-
raster("YOUR_PATH/PRISM_vpdmin_30yr_normal_800mM5_annual_bil/PRISM_vpdmin_30yr_nor
mal_800mM5_annual_bil.bil")
crs(vpdmin1) <- st_crs(4326)$wkt
```

```
vpdmax1 <-
raster('YOUR_PATH/PRISM_vpdmax_30yr_normal_800mM5_annual_bil/PRISM_vpdmax_30yr_nor
mal_800mM5_annual_bil.bil')
crs(vpdmax1) <- st_crs(4326)$wkt

solclear1 <-
raster("YOUR_PATH/PRISM_solclear_30yr_normal_800mM3_annual_bil/PRISM_solclear_30yr
_normal_800mM3_annual_bil.bil")
crs(solclear1) <- st_crs(4326)$wkt

solslope1 <-
raster("YOUR_PATH/PRISM_solslope_30yr_normal_800mM3_annual_bil/PRISM_solslope_30yr
_normal_800mM3_annual_bil.bil")
crs(solslope1) <- st_crs(4326)$wkt

soltotal1 <-
raster("YOUR_PATH/PRISM_soltotal_30yr_normal_800mM3_annual_bil/PRISM_soltotal_30yr
_normal_800mM3_annual_bil.bil")
crs(soltotal1) <- st_crs(4326)$wkt

soltrans1 <-
raster("YOUR_PATH/PRISM_soltrans_30yr_normal_800mM3_annual_bil/PRISM_soltrans_30yr
_normal_800mM3_annual_bil.bil")
crs(soltrans1) <- st_crs(4326)$wkt
```

d. Stack the variables together

```
stacked <- stack(ppt1, tmean1, tmin1, tmax1, tdmean1, vpdmin1,vpdmax1, solclear1,
solslope1, soltotal1, soltrans1)
```

e. Crop the stacked raster to the extent you choose

```
crop <- extent(c(-121, -115, 30, 37.1))

stacked_crop <- crop(stacked, crop)

plot(stacked_crop)
```

f. Write raster files to your computer

```
prism_output_folder <- "YOUR_PATH"

for (i in 1:nlayers(stacked_crop)){
  output_filename <- file.path(prism_output_folder,
paste0(names(stacked_crop[[i]]), ".tif"))
```

```
  writeRaster(stacked_crop[[i]], filename = file.path(output_filename),
format='GTiff', overwrite=T)
}
```

3. Download QGIS

QGIS is an open source GIS program. It does most of what ArcGIS does, but for free! It is available for all major platforms and more. Here is the link to download QGIS for your platform.

https://qgis.org/download/

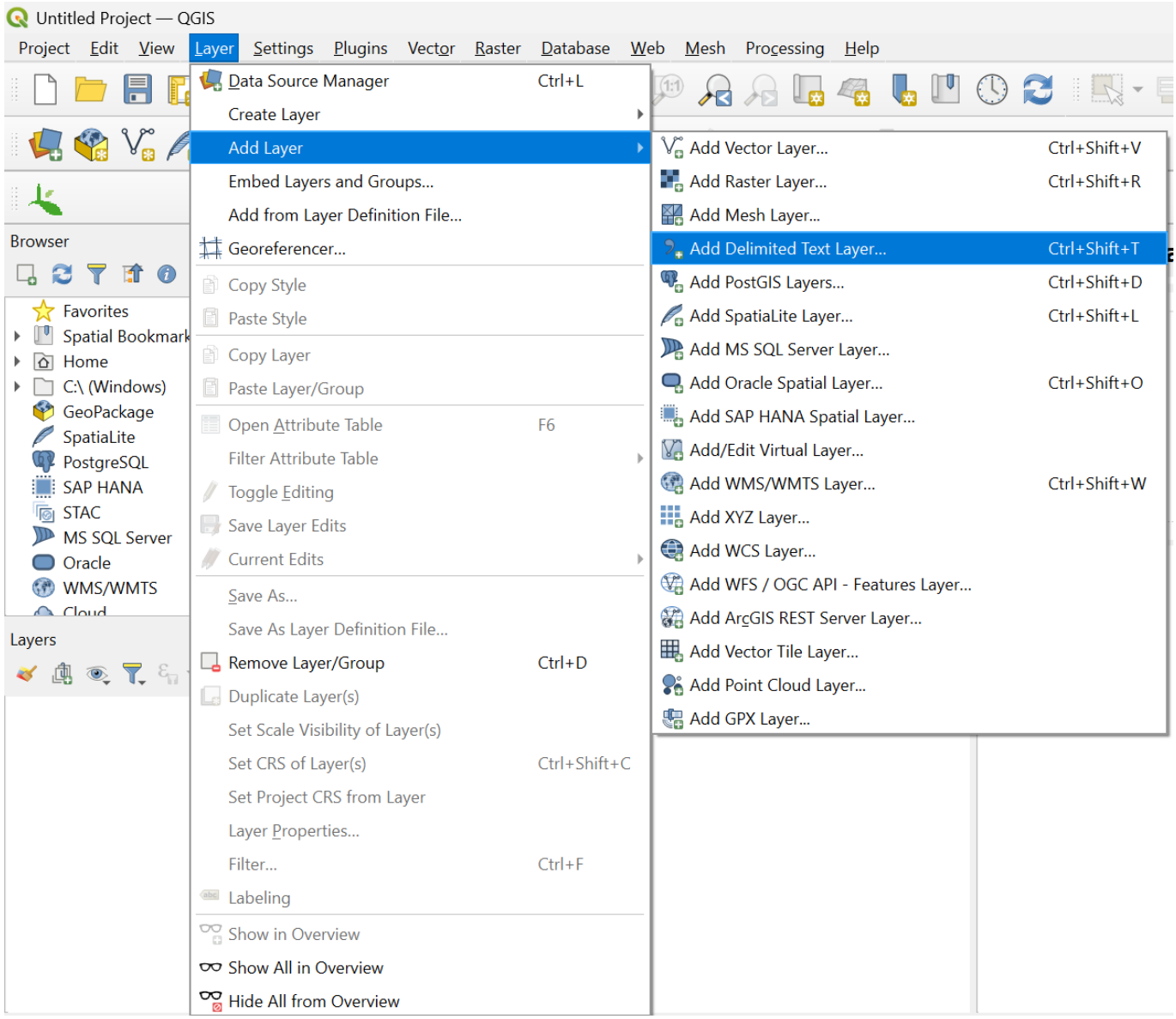
You can just click the "Skip it and go to download" button under the donation banner. For the purposes here, I recommend going with the Standalone installer - it's simpler and works for what we need to do. I tend to use the Long Term Release version - generally fewer bugs.

4. Import data into QGIS

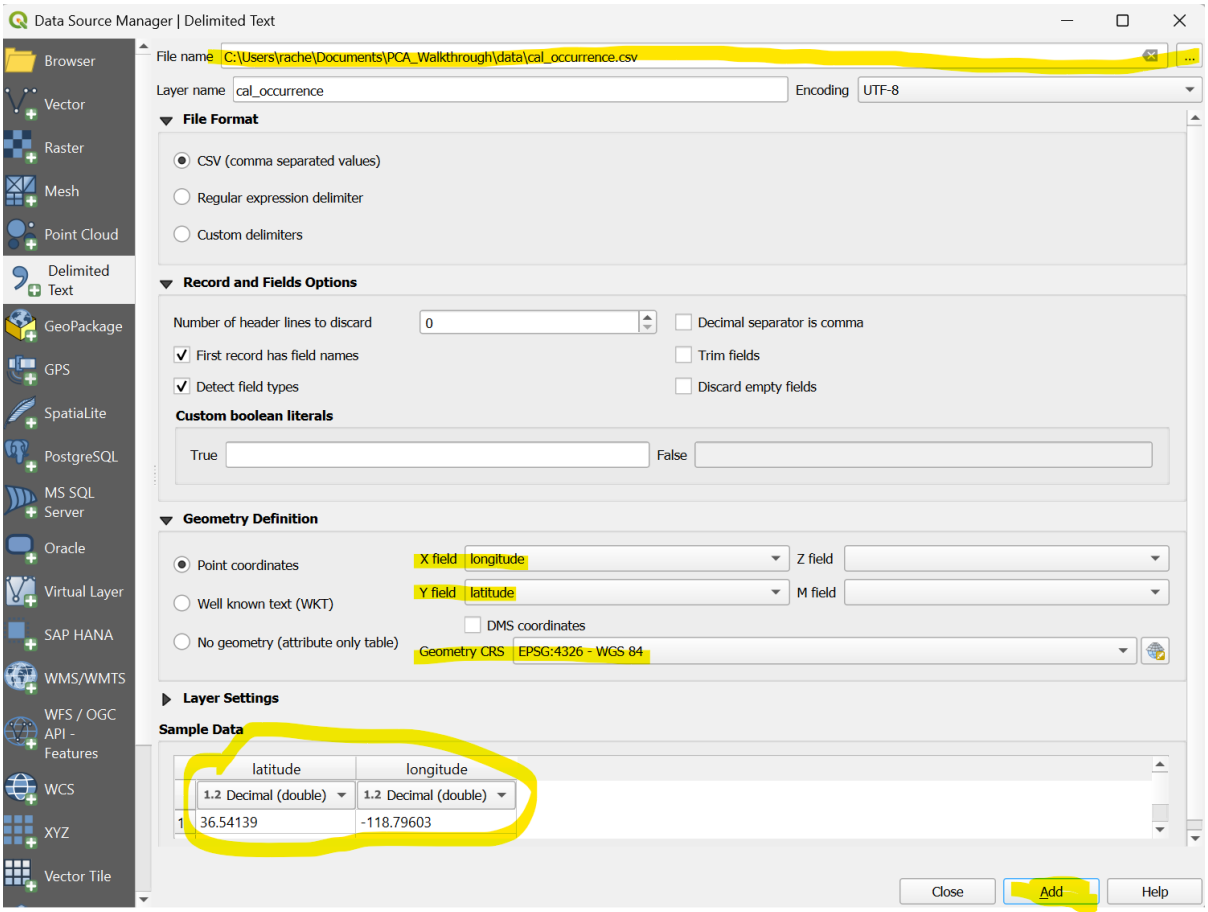
Open QGIS to a new project

a. Add your csv file

1. Go to the top of the screen and selecting *Layer* > *Add Layer* > *Add Delimited Text Layer*...



1. Choose your file at the top "..."
    - Make sure the *X* field says *longitude* and the *Y* field says *latitude*.
    - Make sure the CRS is in the projection you want. Usually WGS 84 - ESPG:4326 is standard.
    - Double-check at the bottom under Sample Data that the latitude, longitude, and anything else look correct.

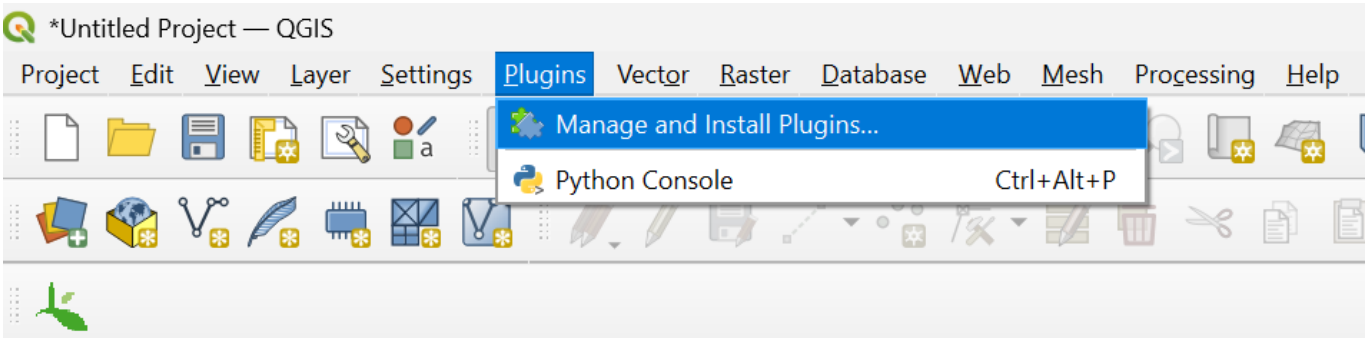


b. Add background map

This is just to verify that your coordinates are in the correct geographic location (i.e. you haven't accidentally swapped the latitude and longitude or mistyped a number or anything like that)

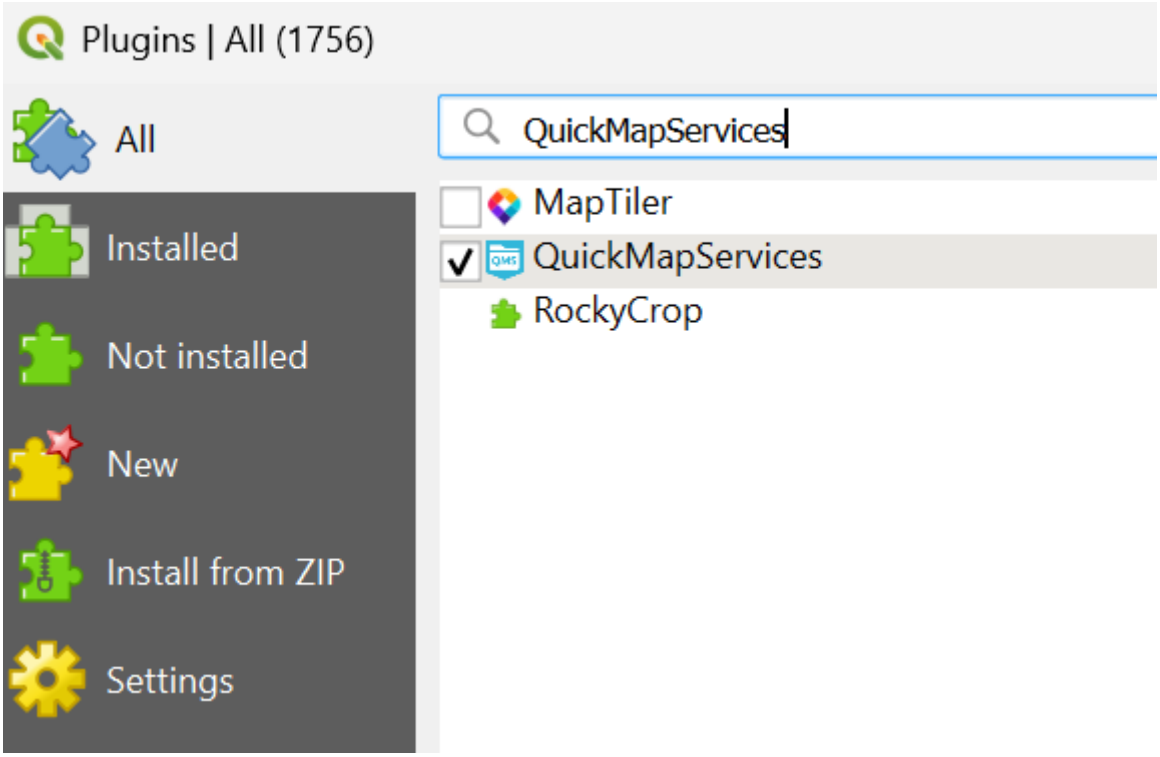
1. Install the QuickMapServices plugin

a. Go to the top toolbar and select **Plugins** > **Manage and Install Plugins...**



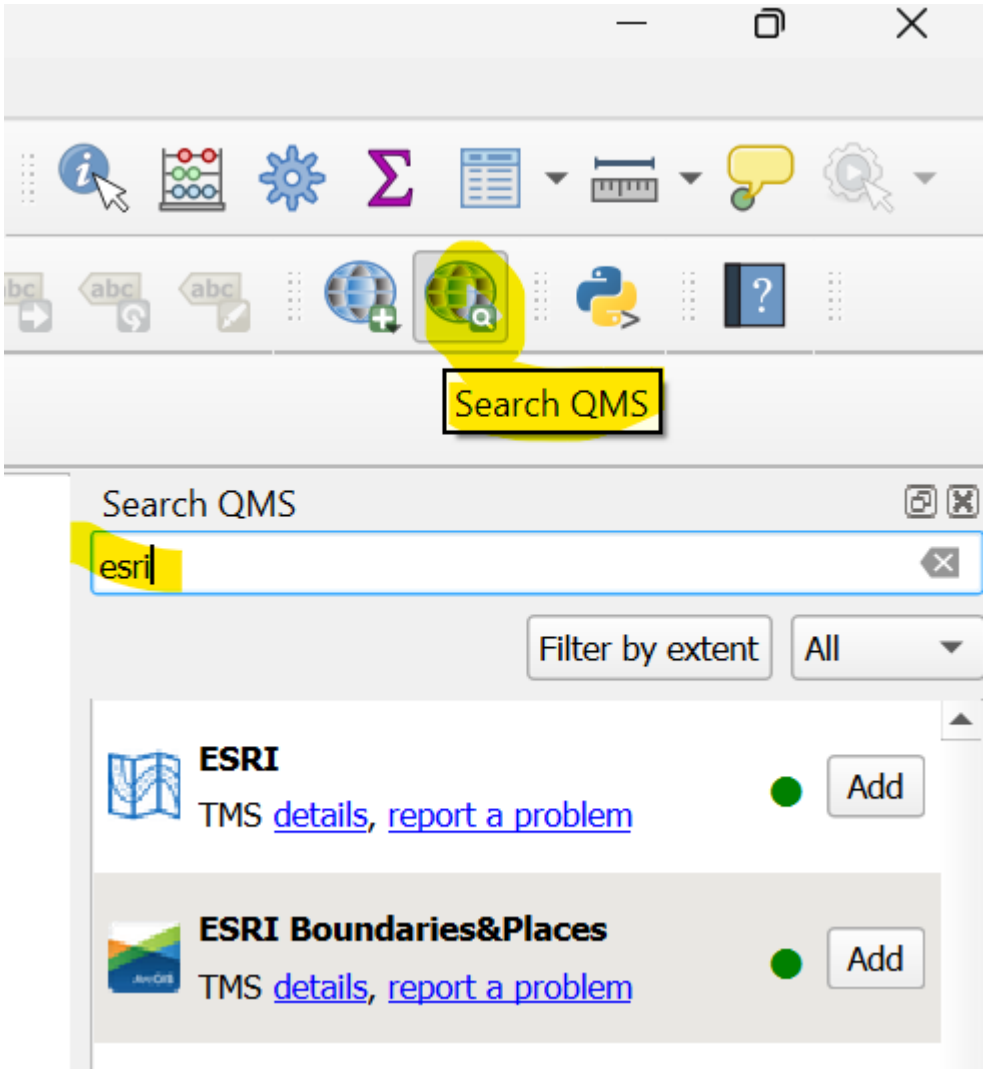
b. Type "QuickMapServices" into the search bar at the top of the window

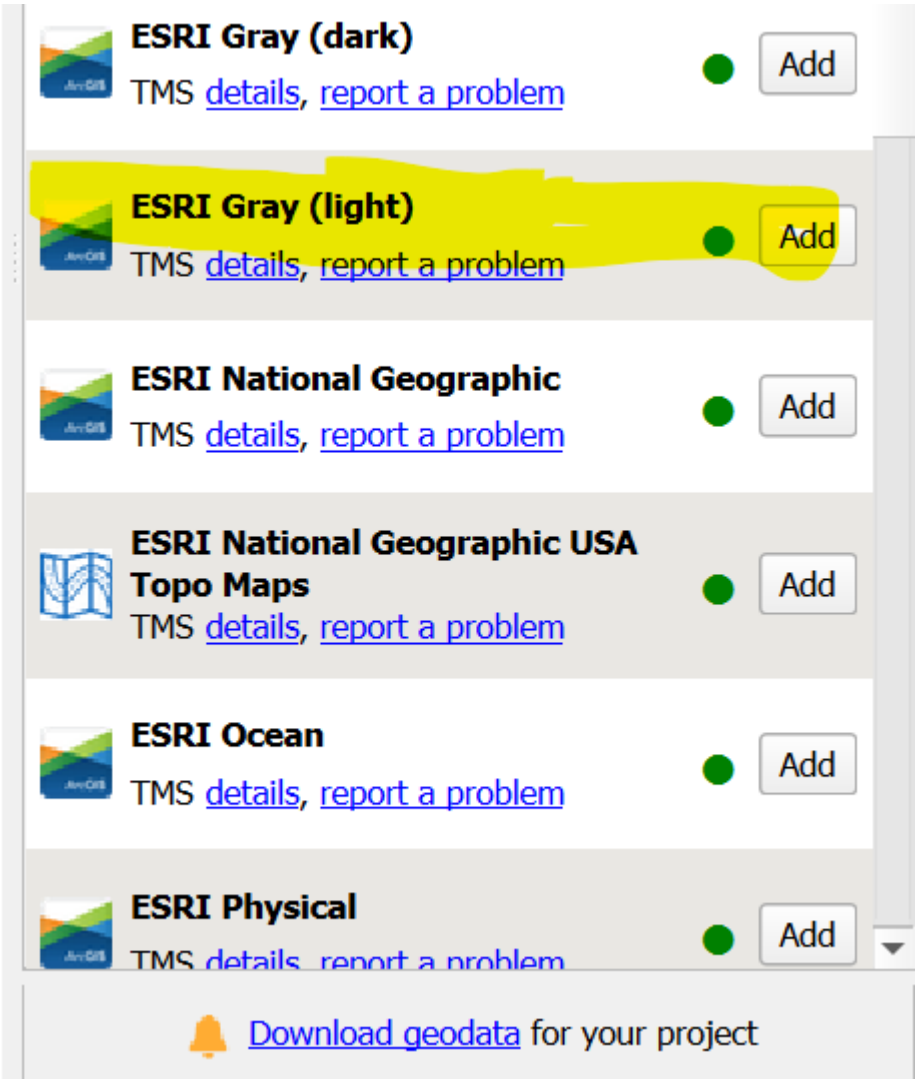
c. Click **QuickMapServices** and click **Install Plugin**



Now there should be two globe icons in a toolbar near the top

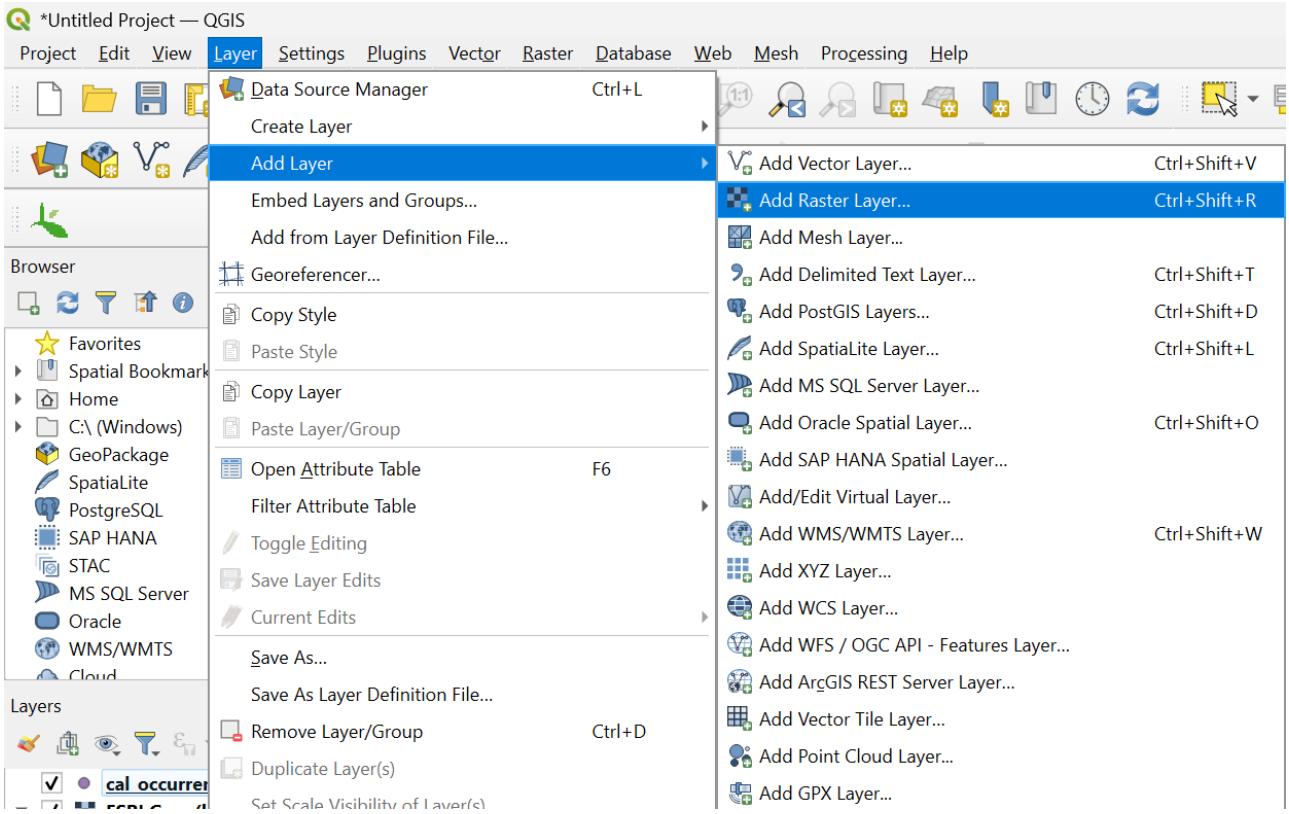
2. Click the **Globe with the magnifying glass**
3. In the sidebar that pops up, type "esri"
4. Choose **ESRI Gray (light)** (Doesn't really matter which map you choose) and hit **Add**





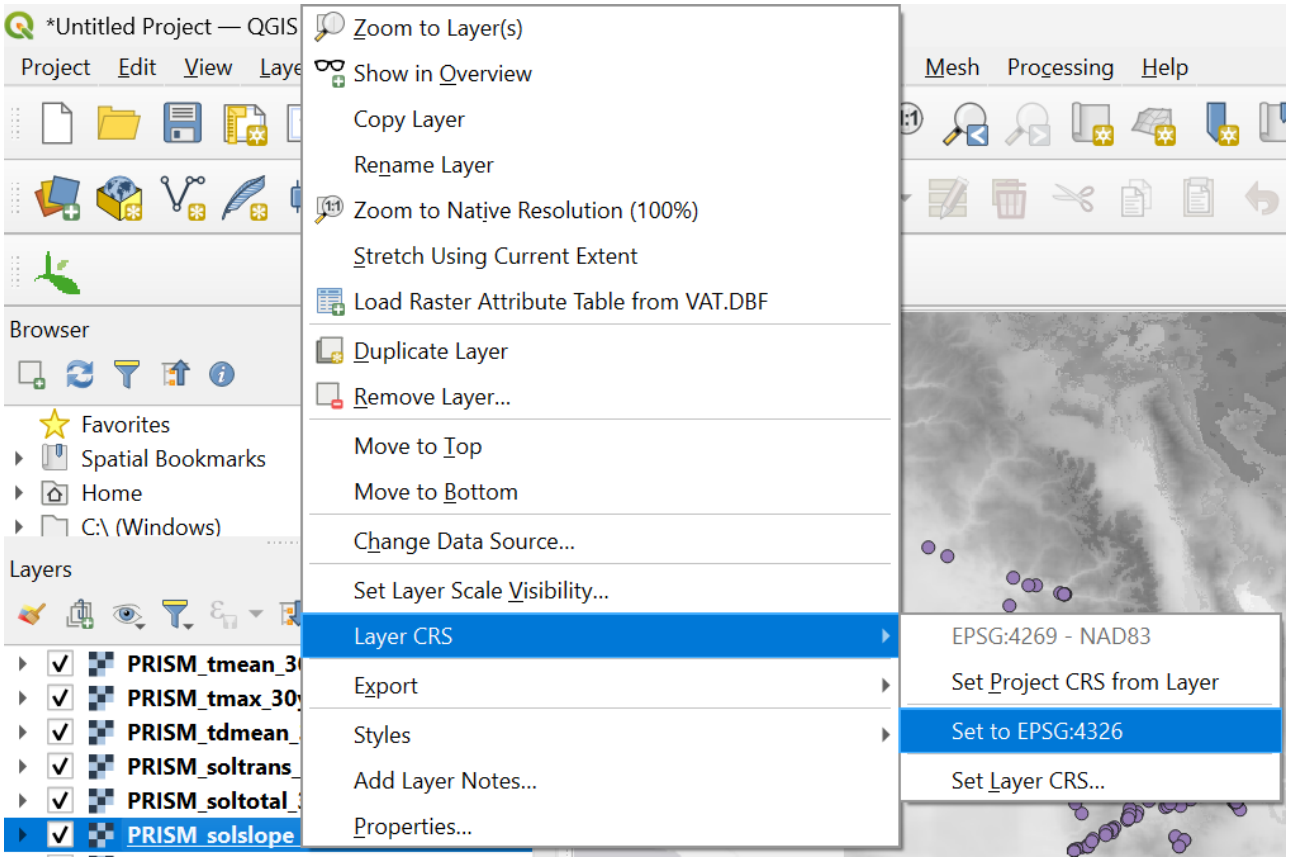
c. Add Raster Layers

1. Select **Layer**>**Add Layer**>**Add Raster Layer...** from the top toolbar



2. Select all rasters in the bioclim folder, and hit **Add** Without closing the panel, go back and do the same for the ISRIC rasters. Then again for the PRISM rasters. Adding the PRISM rasters will result in a window popping up asking about the CRS. We will have to manually set each PRISM layer's CRS anyway, but I just select the one that says "California South of 36 degrees" or something to that effect. Exit this window.

3. Set the CRS of the PRISM rasters by right-clicking each PRISM layer in the layer panel, then selecting **Layer CRS>Set to EPSG:4326**



The PRISM raster will be in a different CRS upon adding them, so this helps keep everything consistent.

5. Extracting Raster Values at Occurrence Points

And now, the moment we've all been waiting for... Getting the environmental data at the occurrence points! 1. Install the **Point sampling tool** plugin by going back to **Plugins>Manage and Install Plugins** then typing "Point sampling tool" into the search bar, selecting **Point sampling tool**, and hitting **Install Plugin**

2. Select the **Point sampling tool** icon from the toolbar (looks like a vertical line with flat blue, yellow, and green objects along it)
3. Make sure your occurrence point vector is selected as *Layer containing sampling points:*
4. Select all the raster layers for which you want data *as well as the latitude and longitude layers of the occurrence vector* (but exclude the base map layer) for *Layers with fields/bands to get values from:*
5. Select a folder and name for the file you want to save the values to. Be sure to change the file type from **Geopackages(*gpkg)*** to **Comma separated values (*.csv*)**
6. Click **OK** Congratulations. You now have a CSV file with the coordinates of your points along with many environmental data for those locations!

6. Running the PCA

a. Load in csv file

```
data <- read.csv('YOUR_PATH/your_file.csv')

# Remove anything that has NAs
data <- na.omit(data)

# Subset the dataframe to remove the latitude and longitude columns, only leaving
the environmental variables and the species as the first column
env_dat <- subset(data, select = -c(longitude, latitude))
```

b. Run the PCA

```
# to run the PCA, we want to select all the environmental variable columns. This
is every column except the first, which is your groups/species. to do this, you
specify that within the env_dat object, you're running prcomp on all rows and
columns 2 - the number of columns. Bracket notation is [rows,columns]. So by
putting nothing before the comma, that's indicating "all rows".
pca <- prcomp(env_dat[,2:ncol(env_dat)], scale=T)

# Now we plot PC1 and PC2 just to take a quick look at it, without distinguishing
the groups
plot(pca$x[,1], pca$x[,2],
     xlab = "PC1",
     ylab = "PC2")
```

c. Percent variation explained by each principal component (PC)

```
# Calculate variation as the standard deviation squared
pca.var <- pca$sdev^2

# Calculate percent variation explained by each PC
pca.var.per <- round(pca.var/sum(pca.var)*100, 1)

# Look at percentages
pca.var.per

# The first number in the list is the percent variation explained by the first PC,
the second number is percent variation explained by the second PC, and so on.

# Visualize the percent variation explained with a scree plot
barplot(pca.var.per, main="Scree Plot", xlab="Principal Component", ylab="Percent
Variation")
```

d. Plot the PCA

```

```

```
# Create an object with the pca scores
pca_scores <- pca$x

# Create an object with the first column of your dataframe, which shows your
species/group
species <- env_dat[, 1]

# Adjust the margins: c(bottom,left,top,right)+x)
par(mar=c(1,1,1,1)+.1)

# Adjust the outer margins
par(oma = c(5,5,0,0))

# plot PC1 and PC2 on a scatter plot. PC1 is denoted by pca_scores[,1], which
means [all rows, 1st column]. PC1 is denoted by pca_scores[,2], which means [all
rows, 2nd column]
plot(pca_scores[, 1], pca_scores[, 2],

     # Color the points according to species. This line is saying if the species
exactly equals "calycinus", make the point purple. If it's not "calycinus", make
the point orange
     col = ifelse(species == "calycinus", "#8B8BDB", "#FF9C00"),

     # Same idea for the shape of the points. It's good to differentiate the point
both by color and shape to make sure everyone can differentiate the points
     pch = ifelse(species == "calycinus", 17, 16),

     # Label the x axis with the words "PC1 - " PLUS the value of percent
variation explained for PC1 PLUS "%" so the axis will read "PC1 - 40.7%"
     xlab = paste("PC1 - ", pca.var.per[1], "%", sep=""),

     # Same thing for the y axis
     ylab = paste("PC2 - ", pca.var.per[2], "%", sep=""),

     # adjust the size of the axis labels and the axes
     cex.lab=1.5, cex.axis=1.5)
# legend("topright",
#        legend = c("CAL", "LON"),
#        col = c("#8B8BDB", "#FF9C00"),
#        pch = c(17,16),
#        bty = "n",
#        pt.cex = 2,
#        cex = 2,
#        text.col = "black",
#        horiz = F ,
#        inset = c(0.1, 0.1))
```

e. Loadings of PCs

Now we look at which environmental variables contribute the most to each principal component axis?

```
# Store PC1 loading scores as an object
loading_scores1 <- pca$rotation[,1]

# Store PC2 loading scores as an object
loading_scores2 <- pca$rotation[,2]

# Get the absolute value of the loading scores for PC1
variable_scores1 <- abs(loading_scores1)

# Sort those values largest to smallest
variable_scores1_ranked <- sort(variable_scores1, decreasing=T)

# Create a list of the 10 largest scores in PC1
top_10_vars1 <- names(variable_scores1_ranked[1:10])

# And again for PC2
variable_scores2 <- abs(loading_scores2)
variable_scores2_ranked <- sort(variable_scores2, decreasing=T)
top_10_vars2 <- names(variable_scores2_ranked[1:10])
```